

LAPORAN UJIAN TENGAH SEMESTER
PEMBUATAN DATA WAREHOUSE
GREEN SUPPLY CHAIN



Disusun Oleh:

Nova Rasyadina Anwar	(2409116092)
Rizky Wahyu Dina Putri	(2409116111)
Elvira Agustin	(2409116109)
Keisya Siti Nafisa Andini	(2409116115)

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN

2026

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR GAMBAR.....	4
BAB I PENDAHULUAN.....	6
1.1 Latar Belakang	6
1.2 Rumusan Masalah	7
1.3 Tujuan.....	7
BAB II PROSES ETL DAN PEMBERSIHAN DATA	8
2.1 Gambaran Umum Dataset	8
2.2 Exctract.....	8
2.3 Transform	10
2.4 Load.....	16
BAB III PEMBUATAN DATA WAREHOUSE	18
3.1 Perancangan Skema Data Wrehouse.....	18
3.2 Persiapan tabel dimensi dan fakta	19
3.2.1 Persiapan dim_product.....	19
3.2.2 Persiapan dim_delivery	20
3.2.3 Persiapan fact_supply_chain	21
3.3 Struktur Tabel Data Warehouse	21
3.3.1 Tabel dim_product, Tabel dim_delivery, & Fact_tabel	21
3.4 Generate SQL dan Ekspor Database	23
3.4.1 Fungsi Pembantu Val() dan df_to_inserts()	23
3.4.2 Pembuatan Database dan Tabel.....	24
3.4.3 Pembuatan View	26

3.4.4 Ekspor ke File SQL	26
3.4.5 Proses Verifikasi	27
BAB IV PENUTUP	30
4.1 Kesimpulan.....	30
4.2 Saran.....	30

DAFTAR GAMBAR

Gambar 1. 1 Import Library	8
Gambar 1. 2 Import File.....	8
Gambar 1. 3 Pengecekan Data	9
Gambar 1. 4 Statistik Deskriptif.....	9
Gambar 1. 5 Kolom Statistik.....	9
Gambar 1. 6 Inkonsistensi Data	10
Gambar 1. 7 Inkonsistensi Data	11
Gambar 1. 8 Hasil Analisis Inkonsistensi Data.....	12
Gambar 1. 9 Handle Missing Values	13
Gambar 1. 10 Handle Duplikat	14
Gambar 1. 11 Handle Outliers.....	14
Gambar 1. 12 Standar Deviasi.....	15
Gambar 1. 13 Hasil Analisis Standar Deviasi	16
Gambar 1. 14 Tahap Load Hasil Data CSV Bersih.....	17
Gambar 2. 1 Pembuatan Skema Data Warehouse.....	18
Gambar 2. 2 Hasil Tabel Skema Dtaa Warehouse	19
Gambar 2. 3 Pembuatan Tabel Dimensi Produk	20
Gambar 2. 4 Pembuatan Tabel Dimensi Delivery.....	20
Gambar 2. 5 Pembuatan Tabel Data Fakta.....	21
Gambar 2. 6 Hasil Tabel Dimensi Produk	22
Gambar 2. 7 Hasil Tabel Dimensi Delivery	22
Gambar 2. 8 Hasil Data Fakta	23
Gambar 2. 9 Hasil Tabel Data Fakta di Database	23
Gambar 2. 10 Konversi Data ke Format Database SQL	24
Gambar 2. 11 Pembuatan Struktur Database Bagian 1	25
Gambar 2. 12 Pembuatan Struktur Database Bagian 2	25
Gambar 2. 13 Pembuatan Tabel View	26
Gambar 2. 14 Pembuatan Struktur Database Berhasil	27
Gambar 2. 15 Verifikasi Data Pada Seluruh Tabel Bagian 1	28

Gambar 2. 16 Verifikasi Data Pada Seluruh Tabel Bagian 2.....	29
Gambar 2. 17 Hasil Verifikasi Data dan Unduh File Database.	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat telah mendorong organisasi dan perusahaan dengan memanfaatkan data sebagai asset strategis dalam pengambilan keputusan. Dalam era persaingan global yang semakin ketat di masa ini, kemampuan suatu perusahaan untuk mengelola dan menganalisis data secara efektif menjadi faktor utama dalam mempertahankan daya saing. Salah satu teknologi yang memungkinkan hal ini adalah data warehouse, data warehouse sendiri merupakan sistem penyimpanan data terpusat yang dirancang khusus untuk mendukung proses analisis dan pelaporan bisnis.

Industri rantai pasok (*Supply Chain*) merupakan salah satu sektor yang sangat bergantung pada ketersediaan data yang akurat dan terbaru. Green Supply Chain menambahkan dimensi keberlanjutan lingkungan ke dalam manajemen rantai pasok konvensional, meliputi aspek pengurangan emisi karbon, efisiensi energi, pengelolaan limbah, serta praktik pengadaan yang ramah lingkungan. Kompleksitas data yang dihasilkan dari proses green supply chain mencakup berbagai variabel mulai dari data pemasok, transportasi, produksi, hingga dampak lingkungan.

Dataset yang digunakan dalam laporan ini mencakup berbagai atribut penting yang memuat informasi mengenai pemasok, produk, rute pengiriman, konsumsi energi, emisi karbon, serta indikator kinerja keberlanjutan lainnya yang relevan untuk analisis mendalam.

Tantangan utama dalam pengelolaan data green supply chain adalah keberagaman format data, inkonsistensi nilai, data yang hilang, serta duplikasi data yang dapat menjadi sangat penting sebagai tahapan pembersihan dan transformasi data sebelum data dimuat ke dalam data warehouse. Laporan ini memaparkan proses perancangan dan Pembangunan data warehouse untuk dataset green supply chain, mencakup seluruh tahapan mulai dari analisis kebutuhan, proses ETL, desain skema data warehouse, hingga implementasi dan pengujian sistem.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana kondisi kualitas data awal pada dataset green supply chain dan permasalahan apa saja yang ditemukan seperti missing value, outliers, duplikasi, dan inkonsistensi data?
2. Bagaimana merancang dan mengimplementasikan proses ETL (Extract, Transform, Load) yang efektif untuk membersihkan dan mentransformasi data green supply chain?
3. Bagaimana merancang skema data warehouse yang sesuai untuk mendukung analisis performa green supply chain multidimensional?
4. Bagaimana hasil Pembangunan data warehouse dapat mendukung proses pengambilan Keputusan berbasis data dalam konteks green supply chain?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah diuraikan sebelumnya, tujuan dari pembuatan laporan dan Pembangunan data warehouse ini adalah:

1. Untuk mengidentifikasi dan mendokumentasikan permasalahan kualitas data yang terdapat dalam dataset green supply chain sebagai dasar pelaksanaan proses pembersihan data.
2. Merancang dan mengimplementasikan pipeline ETL yang mampu mengekstrak, membersihkan, mentransformasi, dan memuat data green supply chain ke dalam struktur data warehouse.
3. Membangun skema data warehouse dengan pendekatan dimensional modeling (*star schema* atau *snowflake schema*) yang optimal untuk mendukung analisis multidimensi green supply chain.
4. Menghasilkan data warehouse yang bersih, konsisten, dan siap digunakan untuk keperluan analisis Business Intelligence (BI) dan pelaporan manajemen.

BAB II

PROSES ETL DAN PEMBERSIHAN DATA

2.1 Gambaran Umum Dataset

Dataset yang digunakan dalam laporan ini adalah Green Supply Chain yang dengan format CSV dan memuat berbagai atribut yang mempresentasikan ekosistem green supply chain, mulai dari informasi pemasok, kategori produk, transportasi, hingga indikator kinerja keberlanjutan seperti emisi karbon karbon, konsumsi energi, dan volume limbah. Seluruh proses pengolahan data dilakukan menggunakan Bahasa pemrograman Python yang di olah di dalam Google Colab.

Terdapat beberapa tahapan yang akan dilakukan untuk membuag data warehouse yaitu melalui proses ETL (Extract, Transform, Load).

2.2 Exctract

Proses dimulai dengan mengimpor library yang diperlukan, kemudian dataset dibaca dari hasil CSV ke dalam struktur dataframe menggunakan library pandas. Setelah data berhasil dimuat, dilakukan eksplorasi awal untuk memahami karakyteristik dari dataset secara menyeluruh sebelum proses pembersihan data dimulai

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import sqlite3
```

Gambar 1. 1 Import Library

```
from google.colab import files
uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving green_supply_chain_dataset.csv to green_supply_chain_dataset (2).csv

Gambar 1. 2 Import File

Eksplorasi awal mencakup pengecekan dimensi data (jumlah baris dan kolom), pemeriksaan tipe data setiap atribut, serta perhitungan statistik deskriptif

seperti nilai rata-rata, median, standar deviasi, nilai minimum, dan nilai maksimum untuk setiap kolom numerik.

```
df = pd.read_csv('green_supply_chain_dataset.csv')
print("Shape:", df.shape)
print("Kolom:", df.columns.tolist())

df

*** Shape: (1029, 16)

Kolom: ['ID', 'Product_Type', 'Raw_Material_Usage_kg', 'Energy_Consumption_kWh', 'Waste_Generated_kg', 'Transport_Distance_km', 'CO2_Emissions_kg', 'Column2', 'Manufacturing_Energy_kWh', 'Renewable_Energy_%', 'Cost_$', 'Column1', 'Delivery_Time_days', 'Sustainability_Score', 'Carbon_Footprint_kg_CO2']

ID Product_Type Raw_Material_Usage_kg Energy_Consumption_kWh Waste_Generated_kg Transport_Distance_km CO2_Emissions_kg Column2 Manufacturing_Energy_kWh Renewable_Energy_% Cost_$ Column1 Delivery_Time_days Sustainability_Score Carbon_Footprint_kg_CO2
0 1 Pharmaceutical 187.102474 372.345525 4.580997 2708.230192 266.072501 NaN 195.022097 77.312801 4096.896464 NaN 2
1 2 Food 117.375385 82.437951 31.484820 685.980543 794.083359 NaN 784.206729 31.899124 989.721788 NaN 4
2 3 Automobile 142.363656 82.065527 49.077714 945.360987 166.372960 NaN 775.047264 16.063118 1321.186084 NaN 2
3 4 Food 185.274882 55.448814 10.315275 2083.074057 653.161326 NaN 214.610271 46.877202 1464.672539 NaN 1
4 5 Food 144.375340 480.425629 39.837161 1562.162306 167.653725 NaN 309.596187 13.990935 381.536655 NaN 10
...
1024 1025 Apparel 188.073413 472.237168 27.242137 3198.741657 944.267873 NaN 661.808184 74.555632 916.405461 NaN 10
1025 1026 Apparel 61.919310 100.661176 36.887651 1336.328443 105.535088 NaN 295.987311 73.857819 3720.328877 NaN 6
1026 1027 Electronics 62.100888 319.260868 18.359295 3248.731888 890.126257 NaN 790.153918 17.781240 803.851214 NaN 6
1027 1028 Apparel 50.480694 456.382748 13.022263 1857.245752 716.042708 NaN 487.869496 13.224842 4025.348019 NaN 3
1028 1029 Electronics 35.215993 176.387447 43.990172 488.473395 225.086327 NaN 372.604276 15.469671 3527.405803 NaN 2
1029 rows x 16 columns
```

Gambar 1. 3 Pengecekan Data

```
print("1. INFO DATASET")
df.info()
print("="*55)

print("\n 2. STATISTIK DESKRIPTIF")
df.describe()

***
1. INFO DATASET
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1029 entries, 0 to 1028
Data columns (total 16 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   ID                   1029 non-null   int64
1   Product_Type        1029 non-null   object
2   Raw_Material_Usage_kg  1029 non-null   float64
3   Energy_Consumption_kWh  1029 non-null   float64
4   Waste_Generated_kg    1029 non-null   float64
5   Transport_Distance_km  1029 non-null   float64
6   CO2_Emissions_kg     1029 non-null   float64
7   Column2              0 non-null      float64
8   Manufacturing_Energy_kWh  1029 non-null   float64
9   Renewable_Energy_%    1029 non-null   float64
10  Cost_$               1029 non-null   float64
11  Column1              0 non-null      float64
12  Delivery_Time_days    1029 non-null   int64
13  Sustainability_Score  1029 non-null   float64
14  Recyclability Score (0-1)  1029 non-null   int64
15  Carbon Footprint (kg CO2)  1029 non-null   int64
dtypes: float64(11), int64(4), object(1)
memory usage: 128.8+ KB
=====
```

Gambar 1. 4 Statistik Deskriptif

2. STATISTIK DESKRIPTIF															
	ID	Raw_Material_Usage_kg	Energy_Consumption_kWh	Waste_Generated_kg	Transport_Distance_km	CO2_Emissions_kg	Column2	Manufacturing_Energy_kWh	Renewable_Energy_%	Cost_\$	Column1	Delivery_Time_days	Sustainability_Score	Recyclability Score (0-1)	Carbon Footprint (kg CO2)
count	1029	1029	1029	1029	1029	1029	0.0	1029	1029	1029	0.0	1029	1029	1029	1029
mean	515.000000	102.399955	274.100596	25.482087	2594.796391	523.084614	NaN	447.143245	44.062090	2495.905916	NaN	7.826540	51.153654	0.522838	501.457776
std	287.181916	55.528066	129.784910	14.987540	1402.318119	275.052671	NaN	262.062252	20.005348	1365.864931	NaN	4.623945	14.727887	0.408721	284.585153
min	1.000000	10.880004	50.060612	1.000570	190.159822	50.898245	NaN	100.289940	10.016870	201.166085	NaN	1.000000	12.000000	0.000000	0.000000
25%	250.000000	55.132305	167.871896	13.201428	1304.652468	265.250981	NaN	272.872071	26.744776	1206.332419	NaN	4.000000	40.140000	0.000000	257.000000
50%	515.000000	106.790204	274.215987	25.428931	2488.141966	510.968033	NaN	444.050334	43.829915	2535.577990	NaN	8.000000	50.000000	1.000000	600.000000
75%	772.000000	152.462489	388.630132	37.681912	3678.221929	761.178273	NaN	629.451768	68.387165	3535.180750	NaN	11.000000	61.620000	1.000000	747.000000
max	1029.000000	198.880808	489.256380	49.978327	4888.872086	980.382785	NaN	799.622477	79.823329	4880.861360	NaN	14.000000	91.530000	1.000000	1000.000000

Gambar 1. 5 Kolom Statistik

Dari eksplorasi awal ditemukan sejumlah permasalahan kualitas data yang perlu ditangani, antara lain keberadaan kolom null sebanyak dua kolom, duplikasi ditemukan sebanyak 29,

2.3 Transform

Langkah pertama dalam tahap transform adalah menangani nilai-nilai yang hilang (*missing value*). Pengecekan dilakukan untuk menghitung jumlah dan persentase missing values pada setiap kolom.

Cek Inkonsistensi Data

```
# CEK INKONSISTENSI DATA
print("=" * 55)
print("    ANALISIS INKONSISTENSI DATA")
print("=" * 55)

inkonsistensi = []

# 1. Cek nilai negatif pada kolom yang seharusnya positif
cols_positif = [
    'raw_material_usage_kg', 'energy_consumption_kwh',
    'waste_generated_kg',    'transport_distance_km',
    'co2_emissions_kg',      'manufacturing_energy_kwh',
    'cost_usd',               'delivery_time_days',
    'sustainability_score',   'carbon_footprint_kg_co2'
]

for col in cols_positif:
    if col in df.columns:
        n = df[df[col] < 0].shape[0]
        inkonsistensi.append({
            'Kolom' : col,
            'Jenis' : 'Nilai Negatif',
            'Jumlah' : n,
            'Status' : 'OK' if n == 0 else f'! {n} baris'
        })

# 2. Cek nilai di luar rentang logis
batas = {
    'sustainability_score' : (0, 100),
    'renewable_energy_pct' : (0, 100),
    'recyclability_score'   : (0, 1),
    'delivery_time_days'    : (1, 30),
}

for col, (lo, hi) in batas.items():
    if col in df.columns:
        n = df[(df[col] < lo) | (df[col] > hi)].shape[0]
        inkonsistensi.append({
            'Kolom' : col,
            'Jenis' : f'Di luar rentang ({lo}-{hi})',
            'Jumlah' : n,
            'Status' : 'OK' if n == 0 else f'! {n} baris'
        })
```

Gambar 1. 6 Inkonsistensi Data

```

# 3. Cek inkonsistensi kategori (spasi tersembunyi / huruf campuran)
cat_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()
for col in cat_cols:
    # Cek jika ada nilai unik yang mirip karena spasi atau kapital
    vals_raw = df[col].astype(str).unique()
    vals_clean = df[col].astype(str).str.strip().str.title().unique()
    selisih = len(vals_raw) - len(vals_clean)
    inkonsistensi.append({
        'Kolom' : col,
        'Jenis' : 'Inkonsistensi Kategori (spasi/kapital)',
        'Jumlah' : selisih if selisih > 0 else 0,
        'Status' : 'OK' if selisih <= 0 else f'!! {selisih} variasi'
    })

# 4. Cek sustainability_category konsisten dengan sustainability_score
if 'sustainability_category' in df.columns and 'sustainability_score' in df.columns:
    def expected_cat(score):
        if score <= 33: return 'Low'
        elif score <= 66: return 'Medium'
        else: return 'High'

    df['_expected_cat'] = df['sustainability_score'].apply(expected_cat)
    mismatch = df[df['sustainability_category'].astype(str) != df['_expected_cat']].shape[0]
    df.drop(columns=['_expected_cat'], inplace=True)
    inkonsistensi.append({
        'Kolom' : 'sustainability_category vs score',
        'Jenis' : 'Inkonsistensi Label vs Nilai',
        'Jumlah' : mismatch,
        'Status' : 'OK' if mismatch == 0 else f'!! {mismatch} baris tidak cocok'
    })

# 5. Cek apakah co2_emissions_kg dan carbon_footprint_kg_co2 tidak saling kontradiksi
# (keduanya seharusnya positif dan tidak jauh berbeda secara proporsional)
if 'co2_emissions_kg' in df.columns and 'carbon_footprint_kg_co2' in df.columns:
    rasio = df['co2_emissions_kg'] / (df['carbon_footprint_kg_co2'] + 1)
    ekstrem = df[(rasio > 10) | (rasio < 0.1)].shape[0]
    inkonsistensi.append({
        'Kolom' : 'co2 vs carbon_footprint',
        'Jenis' : 'Rasio Ekstrem (> 10x atau < 0.1x)',
        'Jumlah' : ekstrem,
        'Status' : 'OK' if ekstrem == 0 else f'! {ekstrem} baris mencurigakan'
    })

# Tampilkan hasil
df_inkon = pd.DataFrame(inkonsistensi)
print(df_inkon.to_string(index=False))

```

Gambar 1. 7 Inkonsistensi Data

```
# Ringkasan
total_masalah = df_inkon[~df_inkon['Status'].str.startswith('OK')]['Jumlah'].sum()
print(f"\n{'='*55}")
if total_masalah == 0:
    print("Tidak ditemukan inkonsistensi data yang signifikan.")
else:
    print(f"Total potensi inkonsistensi: {total_masalah} baris/item")
    print(f"    → Lanjutkan ke Handle Outlier untuk penanganan lebih lanjut.")
print(f"{'='*55}")
```

```
=====
ANALISIS INKONSISTENSI DATA
=====
```

Kolom	Jenis	Jumlah	Status
raw_material_usage_kg	Nilai Negatif	0	OK
energy_consumption_kwh	Nilai Negatif	0	OK
waste_generated_kg	Nilai Negatif	0	OK
transport_distance_km	Nilai Negatif	0	OK
co2_emissions_kg	Nilai Negatif	0	OK
manufacturing_energy_kwh	Nilai Negatif	0	OK
cost_usd	Nilai Negatif	0	OK
delivery_time_days	Nilai Negatif	0	OK
sustainability_score	Nilai Negatif	0	OK
carbon_footprint_kg_co2	Nilai Negatif	0	OK
sustainability_score	Di luar rentang (0-100)	0	OK
renewable_energy_pct	Di luar rentang (0-100)	0	OK
recyclability_score	Di luar rentang (0-1)	0	OK
delivery_time_days	Di luar rentang (1-30)	0	OK
product_type	Inkonsistensi Kategori (spasi/kapital)	0	OK
co2 vs carbon_footprint	Rasio Ekstrem (> 10x atau < 0.1x)	65 !	65 baris mencurigakan

```
=====
Total potensi inkonsistensi: 65 baris/item
    → Lanjutkan ke Handle Outlier untuk penanganan lebih lanjut.
=====
```

Gambar 1. 8 Hasil Analisis Inkonsistensi Data

Pada tahap ini ditemukan hasil dari mencari missing values dan inkonsistensi yang di mana terdapat dua kolom berisi null atau hanya kolom kosong dan inkonsistensi format setiap kolom kategori (huruf besar dan kecil tidak selaras), Tindakan yang dilakukan adalah menghapus kolom kosong dan rename setiap kolom kategori agar format data menjadi konsisten serta menambahkan empat kolom kategori antara lain kolom energy_efficiency_ratio, total_carbon_index, cost_per_km, dan sustainability_category.

```
Handle Missing Values

print("Missing values per kolom:")
print(df.isnull().sum())

... Missing values per kolom:
id          0
product_type 0
raw_material_usage_kg 0
energy_consumption_kwh 0
waste_generated_kg 0
transport_distance_km 0
co2_emissions_kg 0
manufacturing_energy_kwh 0
renewable_energy_pct 0
cost_usd 0
delivery_time_days 0
sustainability_score 0
recyclability_score 0
carbon_footprint_kg_co2 0
dtype: int64

num_cols = df.select_dtypes(include='number').columns.tolist()
num_cols.remove('id')

for col in num_cols:
    missing = df[col].isnull().sum()
    if missing > 0:
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"{col}': {missing} nilai diisi median ({median_val:.2f})")

print("\nTotal missing setelah handling:", df.isnull().sum().sum())

Total missing setelah handling: 0
```

Gambar 1. 9 Handle Missing Values

Gambar di atas merupakan handle missing values yang dilakukan untuk validasi nilai hilang pada data Green Supply Chain, setelah dilakukan handling missing values, total missing values pada data adalah 0.

Setelah melakukan handle missing value, dilakukan pula penghilangan duplikasi pada dataset Green Supply Chain, pada keterangan sebelumnya telah didapatkan 29 duplikasi pada data setelah dilakukan pengecekan, untuk mengatasinya dilakukan handle duplikat pada gambar di bawah.

```
Handle Duplikat

dup_count = df.duplicated(subset=df.columns.difference(['id'])).sum()
print(f"Duplikat ditemukan: {dup_count}")

df = df.drop_duplicates(subset=df.columns.difference(['id']))
print(f"Shape setelah deduplikasi: {df.shape}")

... Duplikat ditemukan: 29
    Shape setelah deduplikasi: (1000, 14)
```

Gambar 1. 10 Handle Duplikat

Perlu diketahui bahwa dalam dataset green supply chain ini tidak terdapat outliers yang sudah di cek sebelumnya atau bernilai 0 outliers pada semua datanya.

```
Handle Outlier (Winsorize - IQR)

cols_outlier = [
    'raw_material_usage_kg', 'energy_consumption_kwh',
    'waste_generated_kg', 'transport_distance_km',
    'co2_emissions_kg', 'manufacturing_energy_kwh',
    'cost_usd', 'sustainability_score',
    'carbon_footprint_kg_co2'
]

print("Outlier sebelum winsorize:")
for col in cols_outlier:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    count = df[(df[col] < lower) | (df[col] > upper)].shape[0]
    print(f" {col}: {count} outlier")

... Outlier sebelum winsorize:
    raw_material_usage_kg: 0 outlier
    energy_consumption_kwh: 0 outlier
    waste_generated_kg: 0 outlier
    transport_distance_km: 0 outlier
    co2_emissions_kg: 0 outlier
    manufacturing_energy_kwh: 0 outlier
    cost_usd: 0 outlier
    sustainability_score: 0 outlier
    carbon_footprint_kg_co2: 0 outlier

for col in cols_outlier:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[col] = df[col].clip(lower=lower, upper=upper)

print("Outlier selesai di-cap (winsorized).")

Outlier selesai di-cap (winsorized).
```

Gambar 1. 11 Handle Outliers

Tahap selanjutnya adalah melakukan pengecekan standar deviasi untuk memahami tingkat variabilitas setiap kolom numerik dalam dataset. Analisis ini menggunakan metrik Coefficient of Variation (CV) yaitu perbandingan antara

standar deviasi dengan nilai rata-rata yang dinyatakan dalam persentase. CV dipilih karena memungkinkan perbandingan variabilitas antar kolom yang memiliki satuan dan skala berbeda. Klasifikasi variabilitas dibagi menjadi tiga kategori, rendah jika CV di bawah 15%, sedang jika CV antara 15% hingga 30%, dan tinggi jika CV di atas 30%.

```
# === CEK STANDAR DEVIASI ===
print("=" * 55)
print("ANALISIS STANDAR DEVIASI")
print("=" * 55)

num_cols = df.select_dtypes(include='number').columns.tolist()
num_cols.remove('id')

stats_std = []

for col in num_cols:
    mean = df[col].mean()
    std = df[col].std()
    cv = (std / mean * 100) if mean != 0 else 0 # Coefficient of Variation

    # Klasifikasi variabilitas berdasarkan CV
    if cv < 15:
        variabilitas = "Rendah"
    elif cv < 30:
        variabilitas = "Sedang"
    else:
        variabilitas = "Tinggi"

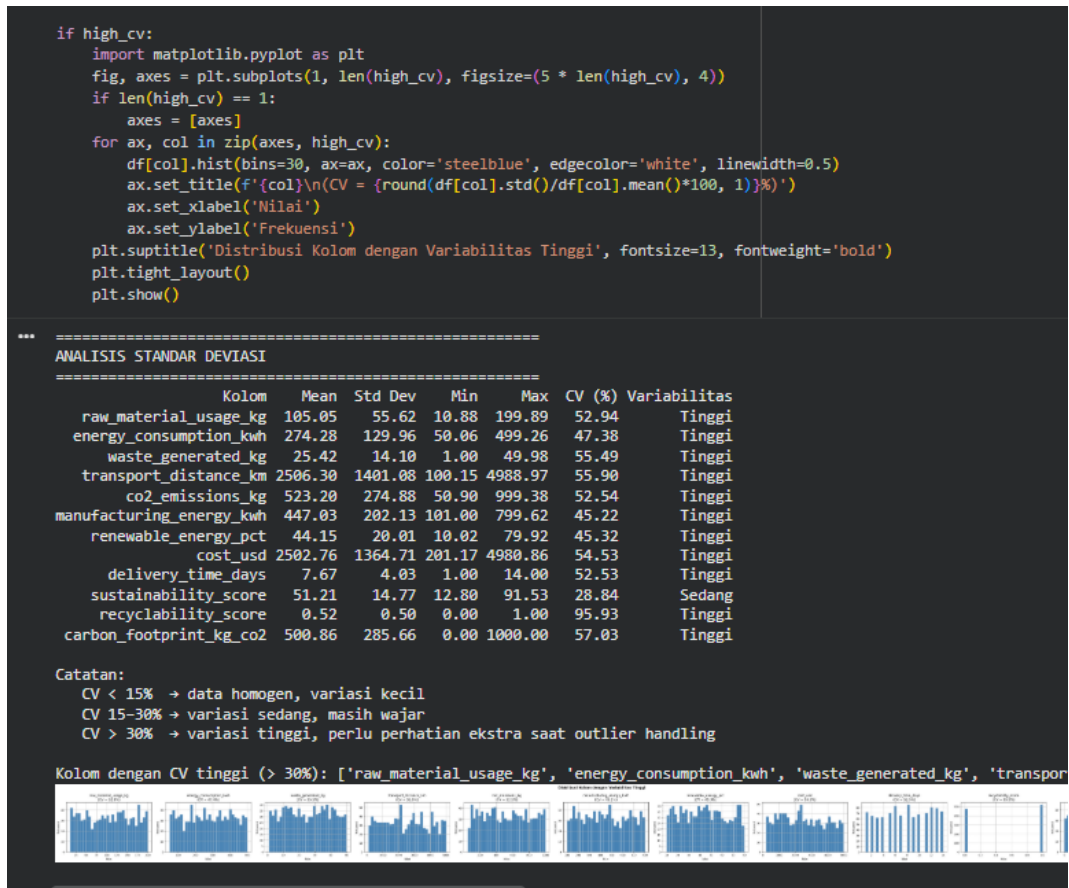
    stats_std.append({
        'Kolom' : col,
        'Mean' : round(mean, 2),
        'Std Dev' : round(std, 2),
        'Min' : round(df[col].min(), 2),
        'Max' : round(df[col].max(), 2),
        'CV (%)' : round(cv, 2),
        'Variabilitas' : variabilitas
    })

df_std = pd.DataFrame(stats_std)
print(df_std.to_string(index=False))

print(f"\nCatatan:")
print(f" CV < 15% → data homogen, variasi kecil")
print(f" CV 15-30% → variasi sedang, masih wajar")
print(f" CV > 30% → variasi tinggi, perlu perhatian ekstra saat outlier handling")

# Visualisasi distribusi kolom dengan std tinggi
high_cv = [row['Kolom'] for row in stats_std if row['CV (%)'] > 30]
print(f"\nKolom dengan CV tinggi (> 30%): {high_cv if high_cv else 'Tidak ada'}")
```

Gambar 1. 12 Standar Deviasi



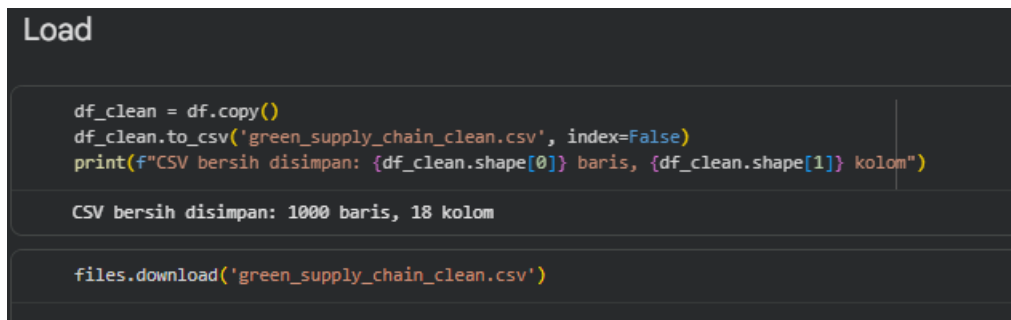
Gambar 1. 13 Hasil Analisis Standar Deviasi

Hasil analisis menunjukkan bahwa hampir seluruh kolom numerik memiliki variabilitas tinggi dengan CV di atas 30%. Kolom `transport_distance_km` dan `waste_generated_kg` menjadi yang paling tinggi variasinya dengan CV masing-masing 55,90% dan 55,49%, diikuti oleh `carbon_kg_co2` sebesar 57,03% dan `recyclability_score` yang sangat tinggi mencapai 95,93%. Satu-satunya kolom yang menunjukkan variabilitas sedang adalah `sustainability_score` dengan CV sebesar 28,84%.

2.4 Load

Setelah seluruh proses pembersihan dan transform selesai, data yang telah bersih dimuat ke dalam struktur data warehouse. Tabel-tabel dimensi dibuat terlebih dahulu karena tabel fakta memerlukan referensi kunci dari dimensi-dimensi

tersebut. Selanjutnya, data fakta beserta seluruh foreign key dimuat ke dalam tabel Fact_Supply_Chain.



```
df_clean = df.copy()
df_clean.to_csv('green_supply_chain_clean.csv', index=False)
print(f"CSV bersih disimpan: {df_clean.shape[0]} baris, {df_clean.shape[1]} kolom")

CSV bersih disimpan: 1000 baris, 18 kolom

files.download('green_supply_chain_clean.csv')
```

Gambar 1. 14 Tahap Load Hasil Data CSV Bersih

Pada tahap ini data bersih sudah jadi dengan output CSV dan akan dilakukan tahap lanjutan dari Load yaitu pembuatan data warehouse dan pembuatan tabel.

BAB III

PEMBUATAN DATA WAREHOUSE

3.1 Perancangan Skema Data Warehouse

Setelah data melalui seluruh proses pembersihan dan transformasi pada tahap ETL dan tersimpan dalam file CSV bersih, langkah selanjutnya adalah membangun struktur data warehouse. Pendekatan yang digunakan adalah star schema, yaitu model data warehouse yang menempatkan satu tabel fakta di tengah yang dihubungkan langsung dengan tabel-tabel dimensi di sekelilingnya. Struktur ini dipilih karena sederhana, mudah dipahami, dan memberikan performa query yang optimal.

Data warehouse yang dibangun terdiri dari satu tabel fakta utama yaitu `fact_supply_chain` yang menyimpan seluruh metrik terukur dari green supply chain, serta dua tabel dimensi yaitu `dim_product` dan `dim_delivery` yang memberikan konteks deskriptif terhadap data fakta tersebut. Selain itu, dibuat pula sebuah view bernama `v_supply_chain_full` yang menggabungkan ketiga tabel tersebut dalam satu tampilan query yang mudah digunakan untuk keperluan analisis.

```
# -- dim_product --
dim_product = (
    df[['product_type', 'sustainability_category']]
    .drop_duplicates(subset='product_type')
    .sort_values('product_type')
    .reset_index(drop=True)
)
dim_product.insert(0, 'product_key', range(1, len(dim_product)+1))
print('dim_product:')
display(dim_product)

# -- dim_delivery --
dim_delivery = (
    df[['delivery_time_days']]
    .drop_duplicates()
    .sort_values('delivery_time_days')
    .reset_index(drop=True)
)
dim_delivery.insert(0, 'delivery_key', range(1, len(dim_delivery)+1))
print('dim_delivery:')
display(dim_delivery)

# -- fact_supply_chain --
df_fact = df.merge(dim_product[['product_key', 'product_type']], on='product_type', how='left')
df_fact = df_fact.merge(dim_delivery[['delivery_key', 'delivery_time_days']], on='delivery_time_days', how='left')

fact_cols = [
    'id', 'product_key', 'delivery_key',
    'raw_material_usage_kg', 'energy_consumption_kwh', 'waste_generated_kg',
    'transport_distance_km', 'co2_emissions_kg', 'manufacturing_energy_kwh',
    'renewable_energy_pct', 'cost_usd', 'delivery_time_days',
    'sustainability_score', 'recyclability_score', 'carbon_footprint_kg_co2',
    'energy_efficiency_ratio', 'total_carbon_index', 'cost_per_km'
]
fact_table = df_fact[fact_cols].rename(columns={'id': 'fact_id'}).reset_index(drop=True)

print(f'\nfact_supply_chain: {fact_table.shape[0]} baris x {fact_table.shape[1]} kolom')
print('Semua tabel siap!')
```

Gambar 2. 1 Pembuatan Skema Data Warehouse

Gambar di bawah menunjukkan output dari pembuatan schema yaitu dim_product dan dim_delivery.

```

... dim_product:
  product_key  product_type  sustainability_category
0           1      Apparel           Medium
1           2    Automotive           Medium
2           3    Electronics           Medium
3           4         Food           Medium
4           5  Pharmaceutical           High

dim_delivery:
  delivery_key  delivery_time_days
0             1                  1
1             2                  2
2             3                  3
3             4                  4
4             5                  5
5             6                  6
6             7                  7
7             8                  8
8             9                  9
9            10                 10
10           11                 11
11           12                 12
12           13                 13
13           14                 14

fact_supply_chain: 1000 baris x 18 kolom
Semua tabel siap!

```

Gambar 2. 2 Hasil Tabel Skema Dtaa Warehouse

3.2 Persiapan tabel dimensi dan fakta

Sebelum tabel-tabel dibuat di database, dilakukan persiapan data terlebih dahulu untuk setiap dimensi dan tabel fakta. Proses ini meliputi pengambilan nilai unik dari kolom-kolom relevan, pemberian surrogate key, sertapenggabungan data untuk membentuk tabel fakta.

3.2.1 Persiapan dim_product

Tabel dimensi produk dibentuk dari kolom product_type dan sustainability_category pada dataset bersih. Duplikasi dihilangkan berdasarkan

kolom `product_type` agar setiap jenis produk hanya muncul satu kali, kemudian data diurutkan secara alfabet. Surrogate key `product_key` diberikan secara beruntun mulai dari angka satu.

```
# — dim_product —
dim_product = (
    df[['product_type', 'sustainability_category']]
    .drop_duplicates(subset='product_type')
    .sort_values('product_type')
    .reset_index(drop=True)
)
dim_product.insert(0, 'product_key', range(1, len(dim_product)+1))
print('dim_product:')
display(dim_product)
```

Gambar 2. 3 Pembuatan Tabel Dimensi Produk

Kode di atas adalah untuk mengambil dua kolom yaitu `product_type` dan `sustainability_category` dari dataframe `beersih`, kemudian menghapus duplikasi berdasarkan `product_type` menggunakan `drop_duplicates()`. Data diurutkan menggunakan `sort_values()` dan index direset agar rapi. Kolom `product key` disisipkan di posisi pertama menggunakan `insert()` dengan nilai berurutan dari 1 hingga jumlah baris dimensi.

3.2.2 Persiapan `dim_delivery`

Tabel dimensi pengiriman dibentuk dari kolom `delivery_time_days` pada dataset bersih. Setiap nilai unik waktu pengiriman dalam hari diambil sebagai satu baris dimensi, kemudian diurutkan dari nilai terkecil ke terbesar. Surrogate key `delivery_key` diberikan secara berurutan mulai dari angka satu.

```
# — dim_delivery —
dim_delivery = (
    df[['delivery_time_days']]
    .drop_duplicates()
    .sort_values('delivery_time_days')
    .reset_index(drop=True)
)
dim_delivery.insert(0, 'delivery_key', range(1, len(dim_delivery)+1))
print('\ndim_delivery:')
display(dim_delivery)
```

Gambar 2. 4 Pembuatan Tabel Dimensi Delivery

Kode di atas mengambil kolom `delivery_time_days` dari dataframe bersih, menghapus nilai duplikat, dan mengurutkannya secara ascending. Surrogate key `delivery_key` kemudian disisipkan di kolom pertama sebagai identifikasi unik setiap baris dimensi pengiriman.

3.2.3 Persiapan `fact_supply_chain`

Tabel fakta dibentuk dengan menggabungkan dataset bersih dengan kedua tabel dimensi menggunakan operasi merge, sehingga setiap baris data fakta memperoleh surrogate key yang sesuai dari masing-masing dimensi. Kolom-kolom yang dipilih untuk tabel fakta mencakup seluruh metrik pengukuran rantai pasok hijau.

```
# -- fact_supply_chain --
df_fact = df.merge(dim_product[['product_key','product_type']], on='product_type', how='left')
df_fact = df_fact.merge(dim_delivery[['delivery_key','delivery_time_days']], on='delivery_time_days', how='left')

fact_cols = [
    'id','product_key','delivery_key',
    'raw_material_usage_kg','energy_consumption_kwh','waste_generated_kg',
    'transport_distance_km','co2_emissions_kg','manufacturing_energy_kwh',
    'renewable_energy_pct','cost_usd','delivery_time_days',
    'sustainability_score','recyclability_score','carbon_footprint_kg_co2',
    'energy_efficiency_ratio','total_carbon_index','cost_per_km'
]
fact_table = df_fact[fact_cols].rename(columns={'id':'fact_id'}).reset_index(drop=True)

print(f'\nfact_supply_chain: {fact_table.shape[0]} baris x {fact_table.shape[1]} kolom')
print('Semua tabel siap!')
```

Gambar 2. 5 Pembuatan Tabel Data Fakta

Kode di atas melakukan dua kali operasi merge secara beruntun, pertama menggabungkan data fakta dengan `dim_product` berdasarkan kolom `product_type`, kemudian dengan `dim_delivery` berdasarkan `delivery_time_days`. Hasilnya adalah dataframe yang sudah memiliki kolom `product_key` dan `delivery_key` sebagai foreign key. Kolom-kolom yang diperlukan dipilih melalui list `fact_cols`, dan kolom `id` diubah namanya menjadi `fact_id` sebagai primary key tabel fakta.

3.3 Struktur Tabel Data Warehouse

3.3.1 Tabel `dim_product`, Tabel `dim_delivery`, & `Fact_table`

Tabel `dim_product` menyimpan informasi jenis produk beserta kategori keberlanjutannya. Setiap baris merepresentasikan satu jenis produk yang unik dalam Green Supply Chain. Kemudian untuk tabel `dim_delivery` yaitu menyimpan

informasi waktu pengiriman dalam hari. Setiap baris merepresentasikan satu nilai waktu pengiriman yang terdapat dalam dataset.

```
*** dim_product:
```

	product_key	product_type	sustainability_category
0	1	Apparel	Medium
1	2	Automotive	Medium
2	3	Electronics	Medium
3	4	Food	Medium
4	5	Pharmaceutical	High

Gambar 2. 6 Hasil Tabel Dimensi Produk

```
dim_delivery:
```

	delivery_key	delivery_time_days
0	1	1
1	2	2
2	3	3
3	4	4
4	5	5
5	6	6
6	7	7
7	8	8
8	9	9
9	10	10
10	11	11
11	12	12
12	13	13
13	14	14

fact_supply_chain: 1000 baris x 18 kolom
Semua tabel siap!

Gambar 2. 7 Hasil Tabel Dimensi Delivery

Kemudian terdapat hasil yang menampilkan bahwa `fact_table` sudah berhasil siap dengan 1000 baris dikali 18 kolom. Tabel `fact_supply_chain` ini sendiri merupakan pusat data warehouse yang menyimpan seluruh metrik pengukuran Green Supply Chain. Tabel ini dihubungkan ke kedua tabel dimensi melalui foreign key `product_key` dan `delivery_key`.

```
fact_supply_chain: 1000 baris x 18 kolom
Semua tabel siap!
```

Gambar 2. 8 Hasil Data Fakta

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	fact_id 🔑	int			No	None			Change Drop More
☐	2	product_key 🔑	int			No	None			Change Drop More
☐	3	delivery_key 🔑	int			No	None			Change Drop More
☐	4	raw_material_usage_kg	double			No	None			Change Drop More
☐	5	energy_consumption_kwh	double			No	None			Change Drop More
☐	6	waste_generated_kg	double			No	None			Change Drop More
☐	7	transport_distance_km	double			No	None			Change Drop More
☐	8	co2_emissions_kg	double			No	None			Change Drop More
☐	9	manufacturing_energy_kwh	double			No	None			Change Drop More
☐	10	renewable_energy_pct	double			No	None			Change Drop More
☐	11	cost_usd	double			No	None			Change Drop More
☐	12	delivery_time_days	int			No	None			Change Drop More
☐	13	sustainability_score	double			No	None			Change Drop More

Gambar 2. 9 Hasil Tabel Data Fakta di Database

3.4 Generate SQL dan Ekspor Database

Setelah seluruh tabel dimensi dan fakta siap dalam bentuk dataframe, langkah selanjutnya adalah menghasilkan skrip SQL lengkap yang mencakup pembuatan database, tabel, pengisian data, dan view. Proses ini dilakukan menggunakan dua fungsi pembantu yang dibuat secara khusus.

3.4.1 Fungsi Pembantu Val() dan df_to_inserts()

Dua fungsi pembantu didefinisikan terlebih dahulu untuk membantu proses konversi dari data frame Python di Google Colab ke format SQL yang valid.

```

def val(v):
    if v is None or (isinstance(v, float) and np.isnan(v)):
        return 'NULL'
    if isinstance(v, str):
        return "'" + v.replace("'", "\'") + "'"
    if isinstance(v, (np.integer, int)):
        return str(int(v))
    if isinstance(v, (np.floating, float)):
        return str(round(float(v), 6))
    return "'" + str(v) + "'"

def df_to_inserts(df, table_name, chunk=500):
    lines = []
    cols = ', '.join([f'`{c}`' for c in df.columns])
    rows = df.values.tolist()
    for i in range(0, len(rows), chunk):
        batch = rows[i:i+chunk]
        val_strs = [f"({', '.join([val(x) for x in row])})" for row in batch]
        lines.append(f"INSERT INTO `{table_name}` ({cols}) VALUES\n" + ',\n'.join(val_strs) + ';')
    return '\n'.join(lines)

```

Gambar 2. 10 Konversi Data ke Format Database SQL

Fungsi `val()` bertugas mengkonversi setiap nilai Python ke Format SQL yang tepat, nilai `None` atau `NaN` dikonversi menjadi `Null`, String diberi tanda kutip Tunggal dengan karakter khusus di escape, bilangan bulat dikonversi ke string integer, dan bilangan decimal dibulatkan hingga 6 angka di belakang koma. Fungsi `df_to_inserts()` memanfaatkan `val()` untuk mengubah seluruh isi data frame menjadi perintah `INSERT SQL` dalam kelompok 500 baris sekaligus, sehingga proses pengisian data berlangsung lebih efisien dan tidak membebani memori.

3.4.2 Pembuatan Database dan Tabel

Skrip SQL dibangun secara bertahap menggunakan list sql yang kemudian digabung menjadi satu string utuh. Proses dimulai dengan pembuatan database `greenchain_db`, diikuti pembuatan tabel dimensi beserta data isisnya, kemudian tabel fakta, dan diakhiri dengan pembuatan view.


```

sql = []
sql.append('-- GreenChain Database')
sql.append('CREATE DATABASE IF NOT EXISTS `greenchain_db`')
sql.append(' CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;')
sql.append('USE `greenchain_db`;')
sql.append('')

sql.append('DROP TABLE IF EXISTS `fact_supply_chain`;')
sql.append('DROP TABLE IF EXISTS `dim_product`;')
sql.append('DROP TABLE IF EXISTS `dim_delivery`;')
sql.append('')

sql.append("""CREATE TABLE `dim_product` (
  `product_key`          INT          NOT NULL AUTO_INCREMENT,
  `product_type`         VARCHAR(100) NOT NULL,
  `sustainability_category` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`product_key`),
  UNIQUE KEY `uq_product_type` (`product_type`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
""")
sql.append(df_to_inserts(dim_product, 'dim_product'))

```

Gambar 2. 11 Pembuatan Struktur Database Bagian 1

```

sql.append('')

sql.append("""CREATE TABLE `fact_supply_chain` (
  `fact_id`              INT          NOT NULL,
  `product_key`          INT          NOT NULL,
  `delivery_key`         INT          NOT NULL,
  `raw_material_usage_kg` DOUBLE      NOT NULL,
  `energy_consumption_kwh` DOUBLE     NOT NULL,
  `waste_generated_kg`   DOUBLE      NOT NULL,
  `transport_distance_km` DOUBLE      NOT NULL,
  `co2_emissions_kg`     DOUBLE      NOT NULL,
  `manufacturing_energy_kwh` DOUBLE   NOT NULL,
  `renewable_energy_pct` DOUBLE      NOT NULL,
  `cost_usd`             DOUBLE      NOT NULL,
  `delivery_time_days`   INT          NOT NULL,
  `sustainability_score` DOUBLE      NOT NULL,
  `recyclability_score`  TINYINT(1) NOT NULL DEFAULT 0,
  `carbon_footprint_kg_co2` INT       NOT NULL,
  `energy_efficiency_ratio` DOUBLE    NOT NULL,
  `total_carbon_index`   DOUBLE      NOT NULL,
  `cost_per_km`          DOUBLE      NOT NULL,
  PRIMARY KEY (`fact_id`),
  KEY `idx_product_key` (`product_key`),
  KEY `idx_delivery_key` (`delivery_key`),
  CONSTRAINT `fk_product` FOREIGN KEY (`product_key`) REFERENCES `dim_product` (`product_key`),
  CONSTRAINT `fk_delivery` FOREIGN KEY (`delivery_key`) REFERENCES `dim_delivery` (`delivery_key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
""")
sql.append(df_to_inserts(fact_table, 'fact_supply_chain'))
sql.append('')

```

Gambar 2. 12 Pembuatan Struktur Database Bagian 2

Perintah Drop Table dijalankan terlebih dahulu sebelum Create Table untuk memastikan proses dapat diulang tanpa konflik jika tabel sudah ada sebelumnya. Tabel dim_product dibuat dengan constraint UNIQUE pada kolom product_type agar tidak ada duplikasi jenis produk. Setelah tabel terbentuk, data langsung dimasukkan menggunakan fungsi df_to_inserts().

3.4.3 Pembuatan View

Selain tabel-tabel utama, dibuat juga sebuah View bernama `v_supply_chain_full` yang menggabungkan tabel fakta dengan kedua tabel dimensi melalui operasi JOIN. View ini memudahkan pengguna dalam melakukan query analitik tanpa perlu menulis JOIN secara manual setiap kali.

```
sql.append("""CREATE VIEW `v_supply_chain_full` AS
SELECT
    f.fact_id, p.product_type, p.sustainability_category,
    f.raw_material_usage_kg, f.energy_consumption_kwh,
    f.waste_generated_kg,    f.transport_distance_km,
    f.co2_emissions_kg,      f.manufacturing_energy_kwh,
    f.renewable_energy_pct,  f.cost_usd,
    f.delivery_time_days,    f.sustainability_score,
    f.recyclability_score,   f.carbon_footprint_kg_co2,
    f.energy_efficiency_ratio, f.total_carbon_index,
    f.cost_per_km
FROM fact_supply_chain f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_delivery d ON f.delivery_key = d.delivery_key;
""")

with open('greenchain_db.sql', 'w', encoding='utf-8') as f:
    f.write('\n'.join(sql))

print('File greenchain_db.sql berhasil dibuat!')
print(f'  dim_product      : {len(dim_product)} baris')
print(f'  dim_delivery       : {len(dim_delivery)} baris')
print(f'  fact_supply_chain   : {len(fact_table)} baris')

** File greenchain_db.sql berhasil dibuat!
   dim_product      : 5 baris
   dim_delivery     : 14 baris
   fact_supply_chain : 1000 baris
```

Gambar 2. 13 Pembuatan Tabel View

View `v_supply_chain_full` menggabungkan seluruh kolom yang relevan dari ketiga tabel menggunakan alias `f` untuk `fact_supply_chain`, `p` untuk `dim_product`, dan `d` untuk `dim_delivery`. Dengan adanya View ini, pengguna cukup melakukan SELECT dari `v_supply_chain_full` untuk mendapatkan data lengkap yang sudah tergabung tanpa perlu menulis query JOIN yang panjang.

3.4.4 Ekspor ke File SQL

Seluruh skrip SQL yang telah dibangun kemudian digabungkan dan diekspor ke file `greenchain_db.sql`. File ini dapat langsung diimpor ke MySQL atau phpMyAdmin untuk membangun data warehouse secara penuh.

```

with open('greenchain_db.sql', 'w', encoding='utf-8') as f:
    f.write('\n'.join(sql))

print('File greenchain_db.sql berhasil dibuat!')
print(f'   dim_product      : {len(dim_product)} baris')
print(f'   dim_delivery      : {len(dim_delivery)} baris')
print(f'   fact_supply_chain : {len(fact_table)} baris')

*** File greenchain_db.sql berhasil dibuat!
    dim_product      : 5 baris
    dim_delivery      : 14 baris
    fact_supply_chain : 1000 baris

```

Gambar 2. 14 Pembuatan Struktur Database Berhasil

Kode di atas menggabungkan seluruh elemen list sql menggunakan join() lalu menuliskannya ke file greenchain_db.sql dengan encoding utf-8. Output akhir berupa informasi jumlah baris pada setiap tabel yang berhasil dimuat, yaitu jumlah baris dim_product, dim_delivery, dan fact_supply_chain, sebagai konfirmasi bahwa seluruh proses pembuatan data warehouse telah berhasil diselesaikan.

3.4.5 Proses Verifikasi

Setelah semua tabel telah berhasil dibuat dan diisi, dilakukan pengecekan secara menyeluruh terhadap data warehouse dengan menampilkan hasil verifikasi dalam bentuk tabel yang memuat nama tabel, jumlah baris, dan status. Pengecekan pertama memastikan jumlah baris pada setiap tabel yang di mana dim_product berisi 5 baris, dim_delivery berisi 14 baris, dan fact_supply_chain berisi 1000 baris, semuanya berstatus OK.

```

#VERIFIKASI DATA WAREHOUSE
print("=" * 55)
print("  VERIFIKASI DATA WAREHOUSE")
print("=" * 55)

verifikasi = []

# 1. Cek jumlah baris tiap tabel
verifikasi.append({
    'Tabel'      : 'dim_product',
    'Jumlah Baris': len(dim_product),
    'Status'     : 'OK' if len(dim_product) > 0 else '! KOSONG'
})
verifikasi.append({
    'Tabel'      : 'dim_delivery',
    'Jumlah Baris': len(dim_delivery),
    'Status'     : 'OK' if len(dim_delivery) > 0 else '! KOSONG'
})
verifikasi.append({
    'Tabel'      : 'fact_supply_chain',
    'Jumlah Baris': len(fact_table),
    'Status'     : 'OK' if len(fact_table) > 0 else '! KOSONG'
})

# 2. Cek tidak ada NULL di kolom kunci
null_product = fact_table['product_key'].isnull().sum()
null_delivery = fact_table['delivery_key'].isnull().sum()
verifikasi.append({
    'Tabel'      : 'fact_supply_chain (product_key)',
    'Jumlah Baris': null_product,
    'Status'     : 'OK' if null_product == 0 else f'! {null_product} NULL'
})
verifikasi.append({
    'Tabel'      : 'fact_supply_chain (delivery_key)',
    'Jumlah Baris': null_delivery,
    'Status'     : 'OK' if null_delivery == 0 else f'! {null_delivery} NULL'
})

# 3. Cek foreign key - semua product_key di fact ada di dim_product
fk_product_invalid = ~fact_table['product_key'].isin(dim_product['product_key'])
verifikasi.append({
    'Tabel'      : 'FK: fact → dim_product',
    'Jumlah Baris': fk_product_invalid.sum(),
    'Status'     : 'OK' if fk_product_invalid.sum() == 0 else f'! {fk_product_invalid.sum()} tidak cocok'
})

```

Gambar 2. 15 Verifikasi Data Pada Seluruh Tabel Bagian 1

Selanjutnya dilakukan pengecekan integritas referensial, yaitu memastikan tidak ada nilai NULL pada kolom foreign key product_key maupun delivery_key di tabel fakta, keduanya menunjukkan nilai 0 yang berarti seluruh baris fakta sudah terhubung dengan benar ke dimensinya. Pengecekan relasi juga dilakukan secara eksplisit untuk memastikan tidak ada foreign key yang merujuk ke baris yang tidak ada di dim_product maupun dim_delivery, dan hasilnya pun 0 alias tidak ada pelanggaran.

```

# 4. Cek foreign key - semua delivery_key di fact ada di dim_delivery
fk_delivery_invalid = ~fact_table['delivery_key'].isin(dim_delivery['delivery_key'])
verifikasi.append({
    'Tabel'      : 'FK: fact → dim_delivery',
    'Jumlah Baris': fk_delivery_invalid.sum(),
    'Status'     : 'OK' if fk_delivery_invalid.sum() == 0 else f'! {fk_delivery_invalid.sum()} tidak cocok'
})

# 5. Cek duplikat di fact_id
dup_fact = fact_table['fact_id'].duplicated().sum()
verifikasi.append({
    'Tabel'      : 'fact_supply_chain (fact_id duplikat)',
    'Jumlah Baris': dup_fact,
    'Status'     : 'OK' if dup_fact == 0 else f'! {dup_fact} duplikat'
})

# 6. Cek konsistensi jumlah - fact harus sama dengan df_clean
selisih = len(fact_table) - len(df_clean)
verifikasi.append({
    'Tabel'      : 'Konsistensi fact vs df_clean',
    'Jumlah Baris': abs(selisih),
    'Status'     : 'OK' if selisih == 0 else f'! selisih {selisih} baris'
})

# Tampilkan hasil
df_verif = pd.DataFrame(verifikasi)
print(df_verif.to_string(index=False))

# Ringkasan akhir
ada_masalah = df_verif[~df_verif['Status'].str.startswith('OK')]
print(f"\n{'='*55}")
if ada_masalah.empty:
    print("Verifikasi Berhasil, Data Warehouse siap digunakan!")
else:
    print(f"! Ditemukan {len(ada_masalah)} item bermasalah - cek tabel di atas.")
print(f"{'='*55}")

```

Gambar 2. 16 Verifikasi Data Pada Seluruh Tabel Bagian 2

Terakhir dilakukan dua pengecekan tambahan, yaitu memastikan tidak ada duplikasi pada fact_id dan memastikan jumlah baris di tabel fakta konsisten dengan jumlah baris di DataFrame bersih df_clean, keduanya menghasilkan nilai 0 yang berarti data warehouse benar-benar bersih dan konsisten. Proses diakhiri dengan munculnya pesan "Verifikasi Berhasil, Data Warehouse siap digunakan!" yang mengkonfirmasi bahwa seluruh struktur dan isi data warehouse sudah dalam kondisi sempurna.

```

***
=====
VERIFIKASI DATA WAREHOUSE
=====
Tabel      Jumlah Baris Status
dim_product      5      OK
dim_delivery     14      OK
fact_supply_chain 1000     OK
fact_supply_chain (product_key) 0      OK
fact_supply_chain (delivery_key) 0      OK
FK: fact → dim_product      0      OK
FK: fact → dim_delivery     0      OK
fact_supply_chain (fact_id duplikat) 0      OK
Konsistensi fact vs df_clean      0      OK

=====
Verifikasi Berhasil, Data Warehouse siap digunakan!
=====

files.download('greenchain_db.sql')
print("selesai!")

selesai!

```

Gambar 2. 17 Hasil Verifikasi Data dan Unduh File Database.

BAB IV PENUTUP

4.1 Kesimpulan

Proyek pembangunan data warehouse Green Supply Chain telah berhasil diselesaikan melalui serangkaian tahapan yang terstruktur. Proses dimulai dari tahap Extract yaitu pengambilan dataset Green Supply Chain dari Kaggle dan eksplorasi data awal untuk memahami karakteristik dataset. Dilanjutkan dengan tahap Transform yang mencakup analisis standar deviasi untuk memahami variabilitas data, penanganan missing values, pendeteksian dan penanganan outlier, standardisasi format data, encoding variabel kategorikal, serta normalisasi data numerik. Pada tahap Load, data bersih disimpan ke dalam file CSV yang siap digunakan sebagai sumber data warehouse.

Tahap pembuatan data warehouse dilakukan dengan merancang skema Star Schema yang terdiri dari satu tabel fakta fact_supply_chain dan dua tabel dimensi yaitu dim_product dan dim_delivery. Seluruh data berhasil dimuat ke dalam struktur tersebut dan diverifikasi dengan hasil yang sempurna. 1000 baris data fakta, 5 jenis produk unik, dan 14 variasi waktu pengiriman, tanpa ditemukan pelanggaran integritas maupun duplikasi data. Data warehouse kemudian diekspor ke file greenchain_db.sql yang siap diimpor ke MySQL.

4.2 Saran

Untuk pengembangan lebih lanjut, data warehouse yang telah dibangun dapat dihubungkan dengan tool Business Intelligence seperti Tableau atau Power BI untuk menghasilkan visualisasi dan dashboard yang lebih interaktif. Selain itu, dimensi yang ada dapat diperluas dengan menambahkan dimensi waktu dan dimensi wilayah agar analisis dapat dilakukan dari sudut pandang yang lebih beragam. Proses ETL juga dapat diotomatisasi menggunakan pipeline terjadwal sehingga data warehouse dapat diperbarui secara berkala tanpa intervensi manual.